

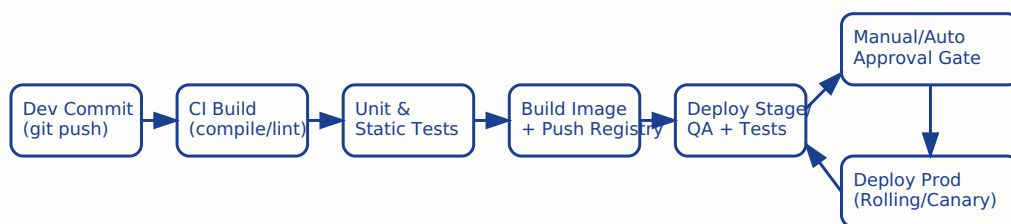
DevOps & SRE

Interview Notes

— written out in full, question by question —

1. CI/CD Pipeline: Commit → Production

The pipeline is a chain of automated gates. Each stage must pass before the next begins.



Stage 1 Source Control — developer pushes to a feature branch; a pull request triggers webhook to CI server (Jenkins/GitLab CI/GitHub Actions).

Stage 2 Continuous Integration — checkout code, install deps, run lint + unit tests + SAST scan (SonarQube). Build fails fast if any gate fails.

Stage 3 Artifact Build — Docker image built with multi-stage build, tagged with commit SHA/semantic version, pushed to registry (ECR/ACR/Harbor). Image scanned for CVEs (Trivy).

Stage 4 Deploy to Staging — CD tool (ArgoCD/Spinnaker/Helm) deploys to a staging namespace;

integration & smoke tests, sometimes DAST security scan.

Stage 5 Approval Gate — for regulated environments, a manual approval; otherwise automatic promotion if staging tests pass.

Stage 6 Production Deploy — strategy chosen (rolling/blue-green/canary), health checks & readiness probes gate traffic shift, followed by post-deploy smoke tests and monitoring.

Stage 7 Observability Loop — metrics/logs/traces feed back into alerting; a bad deploy auto-rolls-back via SLO burn-rate alerts.

2. Deployment succeeded but users get 500 errors

Golden rule: "succeeded" only means the container started — it says nothing about the app being correct.

Step-by-step troubleshooting

1. **Check pod/app status** — `kubectl get pods`, `kubectl describe pod`, `kubectl logs -f <pod>` — look for stack traces, exceptions, crash loops.
2. **Check readiness/liveness probes** — was the pod marked Ready before it could actually serve? Misconfigured probes let broken pods into the load balancer.
3. **Check recent config/env diffs** — compare ConfigMap/Secret/env vars between old and new

deploy; a missing DB connection string is the #1 cause.

4. **Check dependency health** — DB, cache, downstream API. A 500 is often caused by an upstream dependency that changed (schema migration not applied yet, DB pool exhausted).
5. **Check application logs + APM traces** — correlate the 500 timestamp with a trace ID (Jaeger/Datadog APM) to see exactly which line/service threw.
6. **Check the ingress/load balancer layer** — sometimes 500s are actually from the gateway (Nginx/Envoy) misrouting or TLS/cert issues, not the app.
7. **Reproduce locally / in staging** — pull the exact image, run against the same config, confirm root cause before touching prod again.
8. **Mitigate first, root-cause later** — roll back to last known-good version/image tag while the investigation continues in parallel.

Checklist order: **Logs** → **Probes** → **Config diff** → **Dependencies** → **Traces** → **Rollback**

3. Kubernetes pod is continuously restarting

Diagnosis flow

1. `kubectl get pods` → check **RESTARTS** count and **STATUS** (CrashLoopBackOff, OOMKilled, Error).

2. `kubectl describe pod <name>` → read the **Events** section at the bottom — reveals OOMKilled, failed probe, image pull errors, scheduling issues.
3. `kubectl logs <pod> --previous` → logs of the *crashed* container instance (current container may have no logs yet).
4. Check **exit code**:

Exit Code	Meaning
0	Clean exit — app finished/misbehaving entrypoint
1	General application error
137	SIGKILL — usually OOMKilled (memory limit exceeded)
143	SIGTERM — graceful shutdown, maybe probe/timeout related

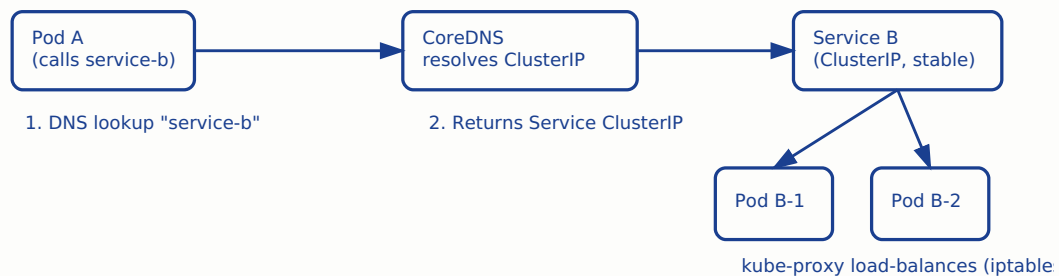
5. If **OOMKilled** → check resource `limits` vs actual usage (metrics-server / `kubectl top pod`); raise limit or fix memory leak.
 6. If **liveness probe failing** → app is slow to start; increase `initialDelaySeconds` or fix a hanging dependency call (DB not reachable at boot).
 7. If **ImagePullBackOff** → wrong tag, private registry auth missing (`imagePullSecrets`).
 8. If **CreateContainerConfigError** → missing ConfigMap/Secret referenced in the pod spec.
 9. Check node-level issues — `kubectl describe node` for disk pressure/memory pressure evictions.
-

4. Kubernetes DNS & service-to-service communication

Every Service gets a DNS entry via **CoreDNS** (runs as pods in `kube-system`). Format:

```
<service-name>.<namespace>.svc.cluster.local
```

Pods get their `/etc/resolv.conf` populated with the CoreDNS ClusterIP as nameserver, plus search domains for the namespace so a short name like `my-service` resolves within the same namespace.



What happens step by step

1. Pod A does `curl service-b` → kernel consults `/etc/resolv.conf` → query sent to CoreDNS ClusterIP.
2. CoreDNS looks up the Service object, returns its stable ClusterIP (not a pod IP).
3. Connection to that ClusterIP hits **kube-proxy** rules (iptables/IPVS) on the node, which load-balances to one of the healthy Pod IPs backing that Service (tracked via Endpoints/EndpointSlices).
4. If using a service mesh (Istio/Linkerd), an Envoy sidecar intercepts the call for mTLS, retries, and circuit breaking before it leaves the pod.

5. Rolling vs Blue-Green vs Canary

Strategy	How it works	Best for	Risk
Rolling	Old pods replaced gradually with new ones, N at a time	Routine releases, no extra infra needed	Both versions run briefly; slow rollback
Blue-Green	Full new environment ("green") built alongside old ("blue"); traffic switched instantly at the LB/DNS	Need instant rollback, zero-downtime cutover	2x infra cost during switch
Canary	Small % of traffic (e.g. 5%) routed to new version, gradually increased while watching metrics	High-risk changes, large user base, need real production signal	Needs strong metrics/automation to be safe

Rule of thumb: Rolling = default/cheap. Blue-Green = need instant, clean rollback. Canary = need to *validate with real traffic* before full exposure.

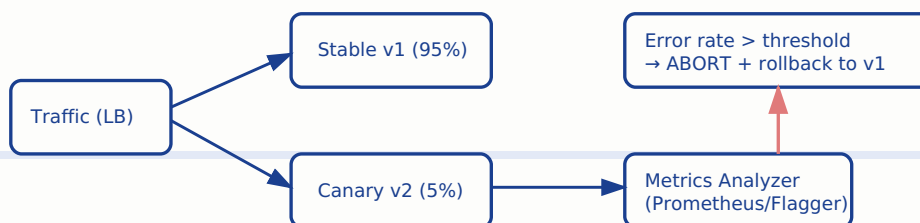
6. Canary deployment — error rate spikes

Prevent it from reaching 100%

1. **Automated analysis, not manual watching** — tools like Argo Rollouts / Flagger continuously query

Prometheus for error-rate & latency SLOs during the canary window.

2. **Define abort thresholds up front** — e.g. error rate $> 1\%$ for 2 minutes triggers automatic rollback.
3. **Freeze traffic increase** — pause the rollout at current %, do not auto-promote to next step.
4. **Auto-rollback** — shift 100% of traffic back to the stable version, scale down canary pods.
5. **Alert on-call** — page with the metrics that tripped the gate (error rate graph, affected endpoints).
6. **Root cause with the isolated canary pods** — since only canary received bad traffic, logs/traces are cleanly isolated to the new version.



7. Terraform drift — infra changed manually

Identify

- Run `terraform plan` — it diffs real cloud state vs the state file and shows drift as pending "changes" even though no `.tf` edits were made.
- Use `terraform plan -refresh-only` to see drift specifically, without proposing to "fix" it yet.

- For scheduled detection, run drift-detection jobs (cron + `terraform plan` in CI, or tools like **driftctl**/Terraform Cloud's drift detection).

Handle it

1. **Decide intent**: was the manual change legitimate (emergency fix) or accidental/unauthorized?
2. **If the manual change should stick** → update the `.tf` code to match reality, then `terraform plan` should show "no changes"; or use `terraform import` / `terraform apply -refresh-only` to reconcile state.
3. **If the manual change should be reverted** → run `terraform apply` normally; Terraform will push the declared config back over the drifted resource.
4. **Prevent recurrence** — restrict console/IAM write access, enforce changes only via pipeline, add drift-detection alerts, use `lifecycle { prevent_destroy = true }` for critical resources.

8. Optimizing a 2GB Docker image

- **Multi-stage builds** — compile/build in a heavy "builder" stage, copy only the final binary/artifacts into a slim runtime stage.
- **Smaller base images** — swap `ubuntu` /full OS images for `alpine`, `distroless`, or slim variants (e.g. `node:20-slim` instead of `node:20`).
- **Minimize layers & clean up** — combine `RUN` commands, remove package manager cache (`apt-`

`get clean` , `rm -rf /var/lib/apt/lists/*`) in the same layer they were created.

- **.dockerignore** — exclude `node_modules` , `.git` , test files, docs from the build context.
- **Avoid dev dependencies in final image** — `npm ci --omit=dev` , don't ship compilers/test frameworks to prod.
- **Layer caching order** — copy dependency manifests (`package.json`, `requirements.txt`) and install deps *before* copying source code, so code changes don't bust the dependency-install cache layer.
- **Compress/scan** — use `docker scout / dive` to inspect layer sizes and find bloat.

Typical result: 2GB → 150-300MB using distroless + multi-stage.

9. CPU/Memory normal but response time increased

When resource usage is normal, the bottleneck is usually somewhere *outside* raw compute.

- **Database** — slow queries, missing index, lock contention, connection pool exhaustion. Check DB slow-query log.
- **Network latency** — DNS resolution delays, cross-AZ calls, TLS handshake overhead, service mesh sidecar hops.

- **Downstream/external API** — a third-party dependency degraded; check dependency latency in APM traces.
- **Thread pool / connection pool saturation** — app-level queueing even though CPU is idle (classic "threads waiting on I/O" scenario).
- **Garbage collection pauses** (JVM/.NET) — GC can spike latency without moving average CPU much.
- **Disk I/O** — slow EBS/PVC, especially if logging synchronously or writing to disk.
- **Queue backlogs** — message queue consumer lag, background job backlog delaying request handling.
- **Noisy neighbor** — another pod/process on the same node stealing I/O or network bandwidth even if CPU shows fine.

Approach: use distributed tracing (Jaeger/Zipkin/Datadog APM) to break down the request timeline span-by-span — find exactly which hop is slow, then dig into that component specifically.

.....

10. Default data source for new Grafana panels

1. Go to **Connections → Data sources** in Grafana UI.
2. Select the data source (e.g. Prometheus).
3. Toggle **"Default"** — this marks it as the data source pre-selected whenever a new panel/dashboard is created.

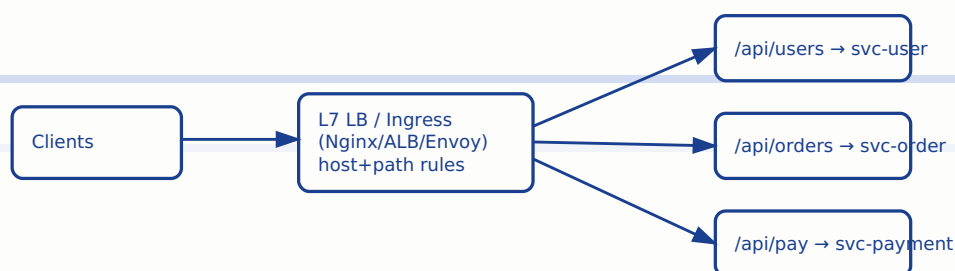
- Only one data source can be default at a time; setting a new one automatically unsets the previous default.
- As-code approach** (recommended for teams): provision it via YAML in `provisioning/datasources/*.yaml`:

```
apiVersion: 1
datasources:
- name: Prometheus
  type: prometheus
  url: http://prometheus:9090
  isDefault: true
```

This keeps the setting reproducible across Grafana restarts/environments instead of a manual UI click.

11. L7 load balancer for multiple microservices

Use path/host-based routing so one endpoint fans out to many backend services.



Key configuration points

- Routing rules** — path-based (`/api/users/*`) or host-based (`orders.example.com`) mapped to each Service/Ingress backend.

- **TLS termination** at the LB, with cert-manager auto-renewing certs.
 - **Health checks** per backend service so the LB stops sending traffic to unhealthy pods.
 - **Rate limiting / WAF rules** at the edge to protect all services uniformly.
 - **Sticky sessions** only where genuinely needed (stateful services).
 - **Observability** — LB access logs + request ID propagation (X-Request-ID) so a request can be traced across services.
 - In Kubernetes this is typically an **Ingress resource** (or Gateway API) backed by an Ingress Controller (Nginx, Traefik, or cloud-native ALB Ingress Controller).
-

12. Managing secrets in Kubernetes

- **Never** hardcode secrets in manifests or images/env vars committed to git.
- **Kubernetes Secrets** — base64-encoded, not encrypted by default; enable **encryption at rest** (EncryptionConfiguration with KMS provider) on the API server.
- **External secret managers** — HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager — store the real secret outside the cluster.
- **External Secrets Operator (ESO)** or **Vault Agent Injector** — syncs secrets from the external manager

into Kubernetes Secrets or injects them directly into pod filesystem/env at runtime.

- **RBAC** — restrict which ServiceAccounts/users can `get/list` Secrets; use least privilege.
- **Sealed Secrets** (Bitnami) — encrypt secrets so they can be safely committed to git as ciphertext, decrypted only by the in-cluster controller.
- **Rotation** — automate credential rotation via the secret manager; avoid long-lived static credentials.
- **Avoid secrets in env vars where possible** — mounted files are safer (env vars can leak via crash dumps/logs); use short-lived tokens where supported (IRSA for AWS, Workload Identity for GCP).

13. Production down at 2 AM — full incident approach

Detection

- Alert fires from monitoring (Prometheus Alertmanager/PagerDuty) based on SLO burn rate or synthetic check failure — on-call paged.

Investigation

1. Acknowledge the page, open the incident channel/bridge, declare severity.
2. Check dashboards first: error rate, latency, traffic, saturation ("the four golden signals").
3. Check recent changes — deployments, config changes, infra changes in the last few hours (correlate timeline).

4. Check dependency health — DB, cache, third-party APIs, cloud provider status page.

Mitigation

- Fastest safe action first: **rollback** if a recent deploy correlates with the incident start.
- If infra-related: scale up, restart unhealthy pods/nodes, failover to standby region/replica.
- If traffic-related: enable rate limiting, shed non-critical load, scale horizontally.
- Communicate status updates at a fixed cadence (e.g. every 15 min) to stakeholders.

RCA (Root Cause Analysis)

- Once stable, write a blameless postmortem: timeline, root cause, contributing factors, what went well, what didn't.
 - Action items with owners and deadlines (e.g. add a missing alert, fix the probe, add a canary gate).
 - Share widely; track action items to closure so the same incident class doesn't repeat.
-

14. Handling sudden 10x traffic

- **Horizontal auto-scaling** — HPA on CPU/memory/custom metrics (requests-per-second) to add pods quickly; Cluster Autoscaler to add nodes when pods can't schedule.

- **Load balancing** — ensure L7 LB and ingress controller itself can scale (avoid a single-instance LB bottleneck).
 - **Caching layers** — CDN for static assets, Redis/Memcached for hot read paths, reduces load hitting the app/DB directly.
 - **Database scaling** — read replicas, connection pooling (PgBouncer), query caching; the DB is usually the first thing to fall over, not the app tier.
 - **Queue-based buffering** — for write-heavy or async-tolerant workloads, push to Kafka/SQS and process asynchronously rather than synchronously blocking requests.
 - **Rate limiting & graceful degradation** — shed non-critical features (e.g. disable recommendations) to protect core checkout/read paths.
 - **Pre-warming** — if the spike is predictable (sale event), pre-scale ahead of time rather than relying purely on reactive autoscaling (which has lag).
 - **Multi-AZ / multi-region** — spread load, avoid single point of failure at the infra level.
-

15. Everything automated, but deployments still fail often

Automation reduces manual error but doesn't guarantee reliability. Investigate:

- **Test coverage gaps** — are unit/integration tests actually catching real regressions, or just passing trivially?

- **Flaky tests** — a noisy test suite trains the team to re-run and ignore failures, masking real issues.
- **Environment parity** — does staging truly mirror production (data volume, config, scaling), or do bugs only appear at prod scale?
- **Missing rollback automation** — does the pipeline auto-detect a bad deploy and roll back, or does it just "fire and forget"?
- **Health checks/gates too shallow** — probes that only check "process is up" rather than real dependency/business-logic health.
- **Poor deployment strategy for risk level** — using plain rolling deploys for high-risk changes instead of canary with metric gates.
- **Secrets/config drift between environments** — automation deploys code fine but config/secrets aren't consistently managed.
- **Team process** — deploys bundling too many changes at once (big-bang releases) instead of small, frequent, easily-attributable ones.
- **Observability gaps** — can't quickly tell *why* a deploy failed, so failures repeat because root cause isn't clearly found each time.

Fix path: tighten test quality → shrink batch size per deploy → add automated rollback on SLO breach → close the environment-parity gap.

— end of notes —